HEdClass – Student Classification System Report

1. Introduction

This project presents the design and implementation of HEdClass, a web-based student classification system developed to address challenges faced by higher education institutions when manually processing complex classification rules under time constraints. Manual processes can lead to risks in accuracy, fairness, and compliance.

The system enables institutional administrators and classification officers to manage student data, academic modules, and classification outcomes in a structured and secure manner.

The primary goal of the system is to automate the classification process while enforcing role-based access control. Institutional administrators are responsible for managing authorised officers and assigning them to degrees/programmes, whereas classification officers manage student records, marks, modules, and classification decisions within their assigned programmes. Additionally, a classification officer can be assigned to more than one degree/programme.

The system was developed using Node.js with Express for backend logic, MySQL for database management, and EJS for dynamic front-end rendering.

2. System Access

GitHub Repository:

https://eeecs.qub.ac.uk/40490439/hedclass

System URL (if hosted):

http://localhost:3000/login

Test Credentials:

Institutional Admin:

- Email: admin@hedclass.com
- Password: admin123

Classification Officers:

- Email: markspencer@mail.com
- Password: markspencer123

- Email: johnjoe@mail.com
- Password: JohnJoe123

3. System Architecture

As shown in Fig. 1, the system follows a Model-View-Controller (MVC) architecture, which separates concerns into distinct components, improving maintainability, readability and scalability.

The Model layer is implemented using a MySQL relational database. It is responsible for storing and retrieving structured data such as users (Institutional administrators and Academic administrators), students, modules, marks, degrees, and assignment relationships.

The View layer is implemented using EJS templates, which allow dynamic rendering of data within HTML pages. This enables the system to display real-time data such as student records, classification results, and dashboard summaries.

The Controller layer is implemented using Express. Controllers handle incoming requests, execute business logic, interact with the database, and return appropriate responses to the user interface.

Authentication is managed using Express session, which maintains user sessions after login. Middleware functions are used extensively to enforce authentication and role-based access control, ensuring that only authorised users can access specific routes.

SQL JOIN operations are used to enforce role-based data filtering at the database level, ensuring officers only access their assigned programmes.

This architecture ensures a clear separation between data handling, business logic, and presentation, making the system easier to extend and maintain. In addition to the system architecture, a use case diagram is provided to illustrate how different users interact with the system and its core functionalities (see Fig. 2).

Although the system is implemented using server-side rendering with EJS templates, it can be extended to incorporate a standalone RESTful API. Such an API would expose key system resources, including students, modules, and degree programmes, through standard HTTP methods (e.g. GET, POST, PUT, DELETE) and return data in JSON format. This would enable integration with external systems or alternative front-end applications, improving the system's flexibility and scalability.

This approach aligns with modern web application design, where front-end and back-end components can be decoupled for improved maintainability.

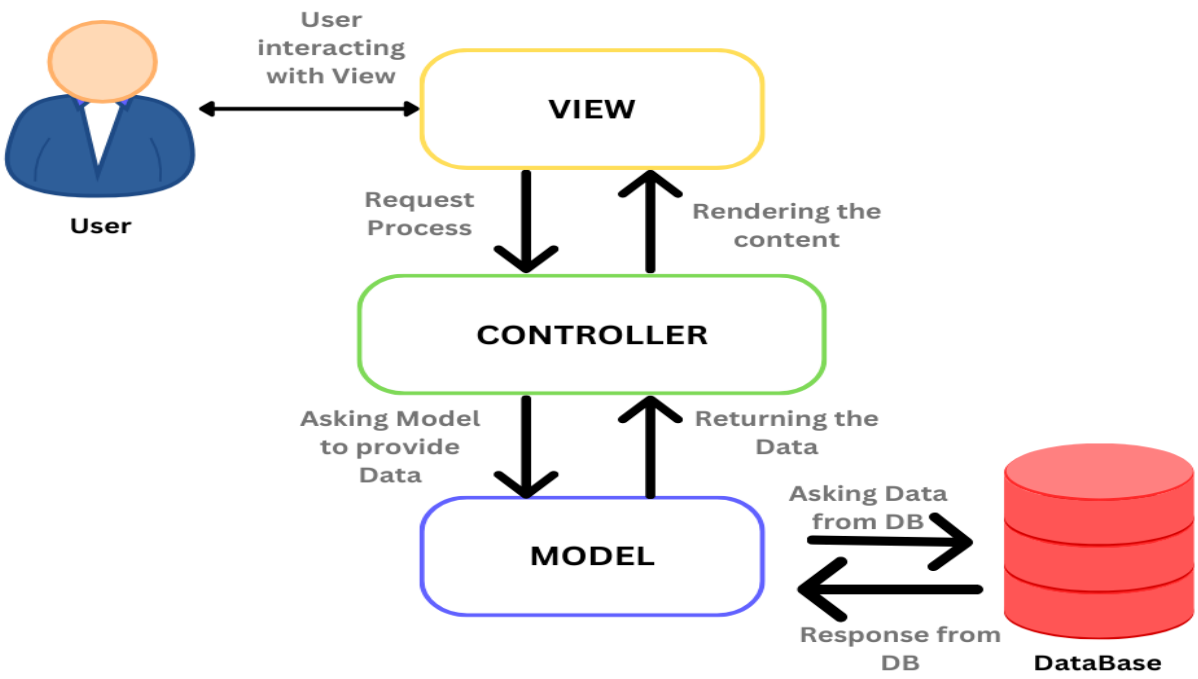

Figure 1: User interacts through a web interface. Requests are handled by Express routes and controllers, which process logic and communicate with the MySQL database.

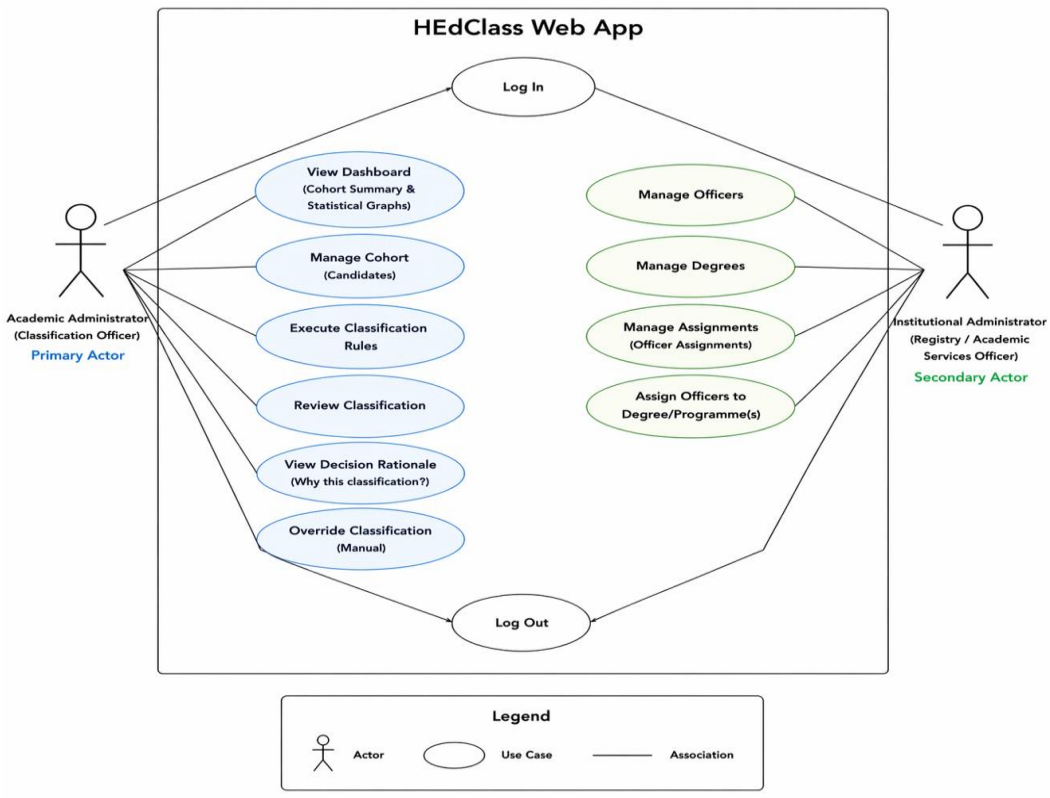

Figure 2: Use Case diagram

The use case diagram illustrates the interactions between the two primary system actors: the Institutional Administrator and the Classification Officer.

The Institutional Administrator is responsible for system configuration tasks, including managing classification officers, assigning officers to degree programmes, and creating degree programmes with associated classification rules.

The Classification Officer interacts with the core functionality of the system. This includes accessing the classification dashboard, managing cohort data (students, modules, and marks), executing classification calculations, and reviewing individual student records. Officers are also able to manually override system-generated classifications and provide a rationale for these decisions.

Additional functionality includes viewing classification distribution through graphical summaries and accessing decision rationale logs for transparency. Optional features such as exporting results and configuring dynamic classification rules further extend system capabilities.

All interactions are governed by authentication, session management, and role-based access control, ensuring that users can only access functionality appropriate to their role.

4. Implemented Functionality Summary

The system implements all required functionality aligned with the specification, covering authentication, administrative operations, and classification processes.

User authentication (A1) allows users to log in using valid credentials and securely log out of the system. Passwords are stored using bcrypt hashing to ensure that sensitive information is not stored in plain text.

The system maintains user state (A2) through session management using Express session. When a user logs in, a session is created and maintained across requests, ensuring that the user remains authenticated while navigating the system.

Role-based access control (A3) is enforced through middleware that verifies user roles before granting access to specific routes. There is one login page and therefore, Institutional Administrators and Classification Officers are redirected to their respective dashboards upon entering the right details, and are restricted to their functionalities, preventing unauthorised access to system features.

Administrative functionality enables the creation and management of classification officers (B1). Institutional Administrators can add, edit, and delete officer accounts, ensuring flexibility in managing system users.

The system supports assigning Classification Officers to specific degree programmes (B2). Each degree programme is assigned to a single officer, ensuring clear ownership.

Classification Officers can also be assigned to multiple programmes (B4), supporting flexible workload distribution.

Institutional Administrators can create and manage degree programmes (B3), including defining base classification rules such as weighting percentages for different academic years.

The classification dashboard provides a cohort summary (C1), displaying information such as total students, degrees, modules and classification distribution. This allows Classification Officers to gain a clear overview of their assigned cohorts.

The system supports full cohort data management (C2), allowing Classification Officers to create, view, update, and delete student records, modules, and marks. This ensures complete control over academic data.

Classification rules are automated (C3), with the system calculating proposed classifications based on defined weighting rules. This ensures consistency and accuracy in classification decisions.

Classification Officers can review and manage individual student records (C4), including overriding system-generated classifications where necessary. This allows for manual intervention in exceptional cases.

A decision rationale log is maintained for each student (C5) providing transparency by recording why a particular classification was assigned. Students may also be flagged for manual review if required.

The system includes a classification distribution summary (C6), presented both in tabular form and through graphical visualisation using Chart.js, enhancing data interpretation.

5. Database Design

The system uses a relational database to manage structured data efficiently. Key tables include users, students, modules, marks, degrees, and officer_degrees.

The officer_degrees table establishes a relationship between Classification Officers and degree programmes, enabling role-based data filtering. Each degree programme is linked to a single Classification Officer, while Classification Officers may be associated with multiple programmes.

Students are linked to degree programmes, and marks are associated with both students and modules. This structure supports efficient querying and ensures data consistency across the system.

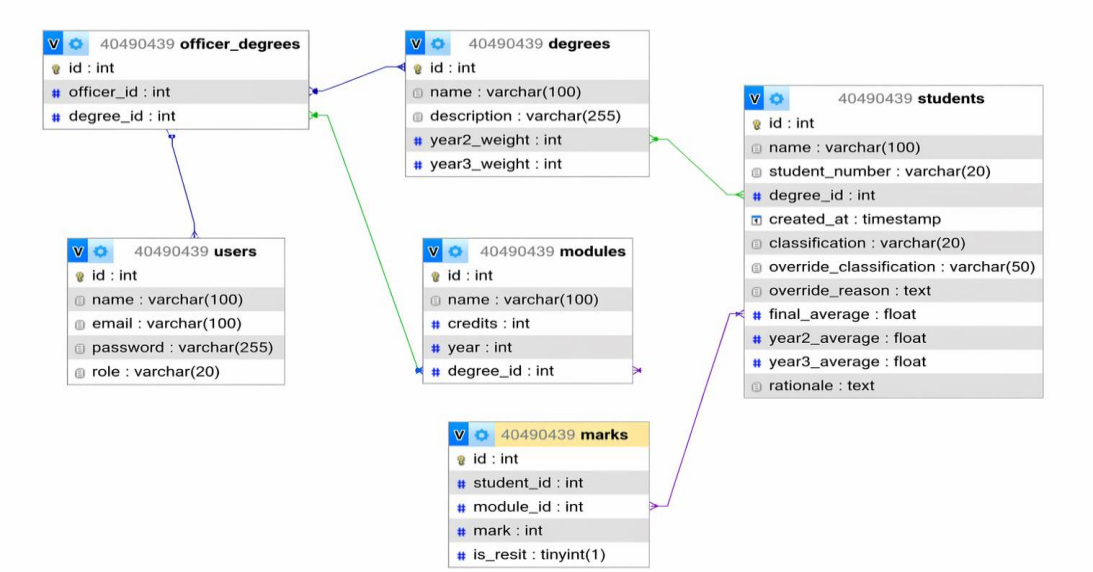

Figure 3: Entity Relationship Diagram

The database is structured using a relational model. The officer_degrees table acts as a mapping table linking Classification Officers (users) to degree programmes, enabling role-based data access.

Each degree is associated with multiple students and modules. Student performance is recorded in the marks table, which links students and modules. This structure supports efficient querying and ensures accurate classification calculations.

6. Non-Functional Requirements

The system satisfies all key non-functional requirements outlined in the specification.

Accuracy (D1) is ensured through deterministic classification logic, where the same input data consistently produces the same classification outcome.

Scalability (D2) is supported through the modular architecture and relational database design, allowing the system to handle increasing numbers of users, programmes, and students.

Security (D3) is implemented through password hashing, session-based authentication, and role-based access control, protecting the system from unauthorised access.

Design (D4) focuses on a modern and responsive user interface, with consistent styling, structured layouts, and user-friendly navigation.

Version control (D5) is maintained using GitLab, ensuring proper tracking of development progress and changes.

7. Challenges and Solutions

Several challenges were encountered during development.

Ensuring correct role-based data access was a significant challenge. Initially, Classification Officers could access all student data. This was resolved by implementing SQL filtering based on Classification Officer assignments.

Managing session persistence and navigation behaviour required careful handling to ensure that users remained authenticated without accessing incorrect pages. This was resolved through proper session management and route protection.

Preventing duplicate degree assignments was also challenging. This was addressed by implementing validation logic to ensure that each degree is assigned to only one officer.

Maintaining consistency in user interface design across multiple pages required the use of reusable CSS structures and layout components.

8. Testing and Validation

The system was tested using a variety of realistic scenarios to ensure reliability and correctness.

Testing included authentication, role-based access control, CRUD operations, classification calculations, and dashboard functionality.

Synthetic test data was generated to simulate real academic environments with multiple students, modules, and classification outcomes.

9. Data Seeding

The system includes an idempotent SQL seeder located in the src/seeder/data.sql

This file:

- Resets the database
- Populates all required data:
    - users
    - degrees
    - students
    - modules
    - marks

The seeder ensures the system can be quickly initialised in a consistent state without manual setup.

The idempotent design allows the script to be run multiple times without creating duplicate data, ensuring reliability and ease of testing.

10. Conclusion

The HEdClass system successfully delivers a structured and secure platform for managing student classification processes. It meets all specified requirements while providing additional features that enhance usability and realism.

The system demonstrates effective use of:

- MVC architecture
- Relational database design
- Role-based access control

By combining automated classification with the ability for manual overrides, the system reflects real-world academic practices and provides a scalable, maintainable solution.